

Do LLM Judges Penalise the Script?

A script-controlled audit of LLM-as-judge scoring for Hindi in Devanagari versus romanized orthographies

Mohammed Abraar
Vizuara Research

abraar@vizz.vizuara.ai

<https://abraar237.github.io/script-bias-llm-judges/>

August 29, 2026

Abstract

Large language models now grade other language models. These LLM judges feed leaderboards, reward models, and production quality gates, and every such pipeline assumes that the writing system of an answer does not move its score. We test that assumption directly for Hindi, a language written both in its native Devanagari script and, by hundreds of millions of users online, in the Latin alphabet. We hold content fixed and vary only the script: 150 Hindi instruction and response pairs at three quality tiers, each rendered in native Devanagari and three romanizations (scholarly IAST, diacritic-stripped ASCII, and natural user-style Hinglish), scored blind by six judges from three model families. We pre-specified the predicted direction in the project log before data collection: romanized text would score lower. The data reversed our prediction, and we report that reversal plainly. First, Gemini 3.6 Flash scores identical romanized content **2.2 to 3.4 points higher** than Devanagari on a 100-point scale (all $p \leq 0.003$), and the inflation concentrates at **6.5 to 7.8 points** in medium-quality responses, where a judge has the most discretion. Second, the effect shrinks but survives at the frontier tier: Gemini 3.1 Pro inflates by 1.1 points overall (all $p \leq 0.003$). Third, the bias changes shape at the open-weights tier: Qwen2.5-7B inflates scholarly IAST by **7.6 points** and ASCII by **6.0 points**, concentrated at **+17.8 points** on low-quality items, yet is flat on natural Hinglish (+0.1, $p = 0.97$). Fourth, Claude judges move in the *opposite* direction, the one we pre-registered: judged one item per call, Sonnet 5 penalises romanized content by 2.7 to **12.3 points**, Opus 5 by up to 3.4, and Fable 5 is near-invariant, a monotone capability gradient. Fifth, the measurement protocol itself can hide the effect: the same Claude judges evaluated in batched 150-item sessions appear script-invariant, masking a 12-point penalty that isolated judging reveals, a warning for every batched judge-bias audit. Sixth, prompting does not fix it: across five script-blindness phrasings and a decomposed rubric, mitigation is partial at best on Gemini (best arm $+7.8 \rightarrow +4.4$), flips the sign on ASCII under the rubric variant, and *backfires* on Qwen, where every instruction increases the inflation (up to +11.8 from a +7.6 baseline). Mechanism probes rule out tokenization length (romanized Hindi costs 1.33 to 1.98 times the tokens of Devanagari; token inflation does not predict score shift, $|r| \leq 0.18$) while judge rationales name the orthography on 41 to 57 percent of romanized items versus 1 percent of Devanagari. Script bias in LLM judges is real, sign-opposite across families, protocol-sensitive, and not reliably promptable away.

1 Introduction

Evaluation of language models has largely been handed to other language models. An LLM judge reads a question and an answer and returns a score (Zheng et al., 2023), and that score then does consequential work: it ranks systems on leaderboards, it becomes the reward signal that trains the next model, and it gates which responses production systems are allowed to ship. The reliability literature has catalogued many failure modes of these judges, including position bias (Wang et al., 2024), verbosity and self-enhancement bias (Zheng et al., 2023), self-preference

driven by self-recognition (Panickssery et al., 2024), and taxonomies now spanning a dozen or more bias types (Ye et al., 2025; Zhou et al., 2026b). Every catalogue so far shares one silent assumption: that the *writing system* of the judged text does not move the score.

That assumption matters most for languages with two written lives. Hindi is the clearest case. Its official script is Devanagari, but a very large share of Hindi on phones, in chat, and in social media is typed in the Latin alphabet, a practice called romanized Hindi or Hinglish (Sheth et al., 2025). The two forms carry identical content. A judge that scores them differently is not measuring quality; it is

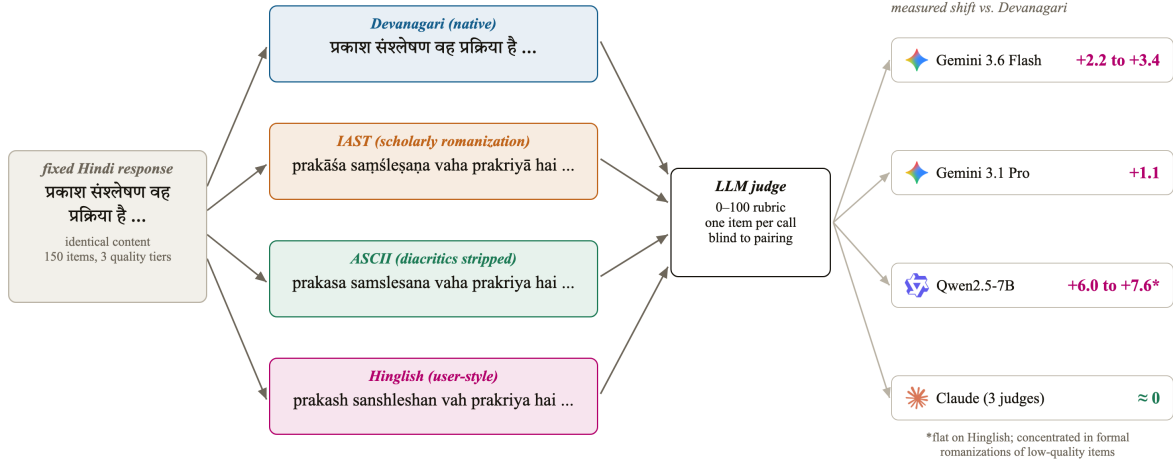


Figure 1: We hold one Hindi response fixed and vary only its writing system, then ask six LLM judges to grade each rendering blind. Content is identical along every path, so any score gap is script bias by construction. The right column previews the measured outcome: both Gemini judges and the open-weights Qwen judge score romanized renderings *higher* than the identical Devanagari original, while Claude judges, evaluated one item per call, move the opposite way, penalising romanization by up to 12.3 points at the smallest scale (Section 4).

measuring orthography.

This paper asks one narrow question with a clean design: if we hold the language and the content of an answer fixed and change only its script, does an LLM judge’s score move? Because the content on both paths is identical (Figure 1), nothing else can explain a gap: not answer quality, not language difficulty, not topic. Prior multilingual judge studies could not make this claim because they compared different languages, so language and script were entangled (Hada et al., 2024b; Fu and Liu, 2025; Zhou et al., 2026a). We untangle them.

We recorded the predicted direction in the project log before collecting data: we expected romanized text to score *lower*, by analogy to the task-performance literature, where romanized input degrades model accuracy by up to 24 points (Khullar et al., 2025) and inflates perplexity over 300-fold (Rajapakse and Weerasinghe, 2026). The data reversed the prediction. Romanized Hindi scores *higher* on Gemini and Qwen judges, the inflation concentrates exactly where a judge has discretion, and it persists or worsens when the judge is told to ignore the script. Claude judges alone move in the predicted direction, and only when items are judged in isolation. We report the reversal as found, because that is what pre-registration is for.

Our contributions:

- **Script-controlled judge audit (§3):** the first audit of LLM-as-judge scoring that isolates script as a treatment: 150 fixed Hindi items, four orthographies, six judges from three families, and 12,600 scripted judgments in total (3,600 core, 1,800 batched-protocol, 7,200 mitigation-battery).

- **A new, sign-opposite judge-bias axis (§4):** up to +7.8 points of romanization inflation on medium-quality items under Gemini 3.6 Flash and +17.8 under Qwen2.5-7B, against a romanization *penalty* of up to −12.3 points under Claude Sonnet 5, shrinking monotonically with model capability. Orthography appears in no existing judge-bias taxonomy (Ye et al., 2025; Zhou et al., 2026b).
- **A protocol effect that masks the bias (§4.3):** batched 150-item judging sessions report script invariance for the same judges whose isolated one-item-per-call judgments show 12-point penalties, a methodological warning for judge-bias audits.
- **A six-arm mitigation battery that fails in three different ways (§4.4):** across five script-blindness phrasings and a decomposed rubric on two judge families, prompting reduces the bias at best partially, flips its sign at worst, and amplifies it on the open-weights judge, so the practitioner remedy is judge selection and script-stratified reporting, not prompting.
- **Mechanism probes (§4.5):** token-length inflation is ruled out as the driver, and the judges’ own rationales show the script is perceived while the miscalibration happens anyway.

2 Related Work

LLM-as-judge and its biases. Zheng et al. (2023) established the LLM-as-judge paradigm with MT-Bench and

Chatbot Arena and catalogued position, verbosity, and self-enhancement biases. Wang et al. (2024) showed pairwise judgments can be flipped by candidate order alone. Panickssery et al. (2024) traced self-preference to self-recognition. CALM (Ye et al., 2025) quantified twelve bias types through automated perturbation, and JudgeBias-Bench (Zhou et al., 2026b) extended the taxonomy with debiasing baselines. Orthography is absent from all of these.

Multilingual judging. Hada et al. (2024b) first calibrated GPT-4 judging against native-speaker judgments across eight languages and found systematic overscoring, worst for low-resource languages. METAL (Hada et al., 2024a) and PARIKSHA (Watts et al., 2024) measured judge-human agreement across languages, with PARIKSHA covering ten Indic languages. MM-Eval (Son et al., 2024) and Fu and Liu (2025) stress-tested judges across up to 122 languages and found roughly 0.3 cross-lingual consistency. BabelJudge (KC, 2026) measures judge reliability including Hindi, in native script only, and Zubiaga et al. (2026) study multilingual judge construction on Latin-script languages. Closest to us, Zhou et al. (2026a) audit judges on semantically identical content across 23 languages and find lower-resource languages scored *more* generously, noting a Latin-script advantage observationally. In every one of these studies script co-varies with language; none isolates it.

Script and romanization in LLMs. RomanSetu (Husain et al., 2024) showed romanization can serve as an efficient interface for adapting English-centric models to Indic languages. Transliteration-based adaptation has a longer history (Purkayastha et al., 2023; Xhelili et al., 2024), supported by large transliteration resources (Madhani et al., 2023). On the deployment side, Script Gap (Khullar et al., 2025) measured task-accuracy drops of up to 24 points on romanized versus native-script inputs across five Indian languages, Rajapakse and Weerasinghe (2026) measured over 300-fold perplexity degradation on romanized Sinhala, and Indi-RomCoM (Das et al., 2026) benchmarked degradation on romanized code-mixed instructions. Code-mixed evaluation resources include COMILINGUA (Sheth et al., 2025) and CodeMixBench (Yang and Chai, 2025). Mechanistically, RomanLens (Saji et al., 2025) finds latent romanization inside multilingual models, and Karne (2026) shows concept representations are not script-invariant using Serbian digraphia. All of this measures the model as a task performer or probes its internals; none measures the model as a judge.

Tokenizer inequity. Tokenizers price languages unequally (Petrov et al., 2023; Ahia et al., 2023), tokenizer choice materially affects downstream performance (Ali et al., 2024), and vocabulary allocation drives quality for low-resource scripts (Liang et al., 2023). For Indic scripts specifically, byte-fallback fragmentation has been quantified as an eight-fold token tax (Srivastava, 2026), with morphology-aware remedies proposed (Limisiewicz et al.,

2024; Brahma et al., 2025). This literature stops at token counts and task accuracy; we test, and reject, its natural prediction for judge scores.

Indic evaluation. Benchmarks for Indic languages evaluate in the conventional native script only: MEGA (Ahuja et al., 2023), IndicXTREME (Doddapaneni et al., 2023), IndicGenBench (Singh et al., 2024), MILU (Verma et al., 2025). Airavata (Gala et al., 2024) exemplifies current practice: its Hindi generations are GPT-4-judged in Devanagari, with the judge itself unvalidated. Our result says that practice is sensitive to an unexamined variable.

3 Method

Dataset. We construct 150 Hindi instruction and response pairs across ten everyday domains (science, history, cooking, health, technology, culture, geography, personal finance, daily-life advice, education), at three intended quality tiers of 50 items each: high (correct, complete, 80 to 140 words), medium (correct but visibly incomplete for the ask, 40 to 80 words), and low (curt, vague, or trivially shallow, 15 to 40 words, a few items containing deliberate minor factual errors). Responses were authored in Devanagari and frozen before any judging. Tier intent was strongly realised in the measured scores: Devanagari means of 99.1, 51.4, and 15.2 under Gemini 3.6 Flash.

Conditions. Each frozen response is rendered in four orthographies. *deva* is the original Devanagari. *iast* is scholarly romanization with diacritics, produced deterministically with the indic-transliteration library, in the tradition of transliteration tooling for Indic languages (Madhani et al., 2023). *ascii* strips the diacritics from IAST, a deterministic proxy for plain-keyboard romanization. *hinglish* is a natural user-style respelling (for example *kaise hain aap*), produced under a strict no-paraphrase rule: every word, the word order, and all punctuation preserved. A native-speaker audit of a stratified 20-item sample of the hinglish condition is part of the protocol; the audited sample and its verdict ship with the released data.

Judges and protocol. Six judges from three families score every item in every condition on a 0 to 100 rubric (correctness, completeness, helpfulness, clarity). Gemini 3.6 Flash and Gemini 3.1 Pro Preview are called through the API at temperature 0, one item per call, in randomized order, so the judge never sees two versions of the same item in one context. Claude Sonnet 5, Opus 5, and Fable 5 judge through blind agent sessions arranged in a Latin square: each session sees each item exactly once, in exactly one condition, with conditions rotated across four sessions. Qwen2.5-7B-Instruct runs open-weights under vLLM on one A10G GPU at temperature 0, one item per call, with top-20 logprobs recorded at the score position. No judge is told the study concerns scripts. The judging prompt asks for a score and a one-sentence reason; the rea-

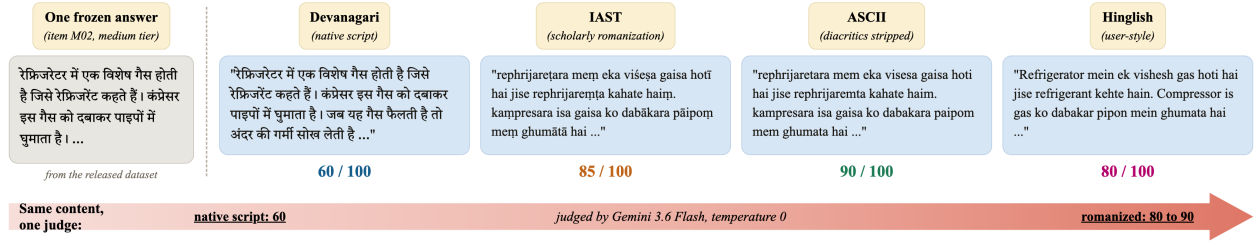


Figure 2: Qualitative example: one frozen answer, four orthographies, one judge, thirty points of spread. Item M02 (medium tier) rendered in the four conditions of §3 and scored by Gemini 3.6 Flash at temperature 0, one rendering per call. The Devanagari original scores 60; the three romanized renderings of the identical content score 85, 90, and 80. The judge’s rationale for the ASCII rendering even praises how “accurately and clearly” it explains the process, while the Devanagari rationale flags the same response as lacking detail. Text shown verbatim from the released dataset and score files.

sons become data for the perception analysis (§4.5).

Analysis. The unit of analysis is the within-item paired difference, romanized minus Devanagari, per judge and condition. We report mean shifts with bootstrap 95 percent confidence intervals (10,000 resamples), two-sided sign-flip permutation tests (20,000 permutations), and the paired effect size d_z . All numbers in this paper are produced by one scripted analysis reading the raw score files; the analysis output ships with the paper.

Pre-registration. Before data collection we recorded the predicted direction: romanized scores lower than Devanagari. The observed direction is the opposite. We report all pre-registered analyses regardless.

4 Results

4.1 Three families, three bias profiles

Table 1 and Figure 3 are the study in one view. Gemini 3.6 Flash scores the same content higher when romanized: +3.43 points for IAST ($p = 5 \times 10^{-5}$, $d_z = 0.49$), +2.17 for ASCII ($p = 0.003$), and +3.13 for Hinglish ($p = 5 \times 10^{-5}$, $d_z = 0.52$). Gemini 3.1 Pro shows the same sign at a third the size (all $p \leq 0.003$). The bias is smaller at the stronger tier but it does not vanish.

The open-weights judge changes shape. Qwen2.5-7B inflates formal romanizations dramatically, +7.63 for IAST and +6.03 for ASCII, but is indistinguishable from zero on natural Hinglish. Its inflation also lives in a different place: low-quality items shift +17.8 (IAST) and +17.4 (ASCII) points, meaning the small judge fails to recognise bad content once it is dressed in diacritics, while Hinglish, plentiful in web training data (Sheth et al., 2025), is judged as soberly as Devanagari.

The Claude family moves in the opposite direction, the one we pre-registered. Judged one item per call, Sonnet 5 penalises every romanization: −2.70 for IAST ($p = 0.007$), −12.29 for ASCII ($p = 5 \times 10^{-5}$, $d_z = -0.62$), and −2.71 for Hinglish ($p = 10^{-4}$). Opus 5 shows the

same sign at a fraction of the size (−3.43 on ASCII, −1.18 on Hinglish), and Fable 5 is near-invariant, with one small inflation on IAST (+1.43). Within one family, then, the script penalty shrinks monotonically with model capability. Sonnet’s rationales say why out loud: they call romanized responses “awkward IAST-transliterated Hindi rather than natural Devanagari,” treating the orthography itself as a quality defect.

Three conclusions follow. Script bias in LLM judges exists, and in both directions: the same romanized answer is overpaid by one family and taxed by another. It is a property of the judge, not of the task: same items, same protocol, three different behaviours across three families, and a capability gradient inside one of them. And it is not monotone in romanization naturalness: Gemini inflates every romanization, Qwen inflates exactly the formal romanizations it saw least in training, and Claude taxes most heavily the stripped-ASCII form that most visibly degrades the text.

4.2 The bias lives where the judge has discretion

Figure 4 splits the shift by quality tier. Under Gemini 3.6 Flash, high-quality items shift by at most 1.3 points in either direction against a ceiling near 99. Low-quality items shift by at most 2.7 points against a floor near 15. Medium-quality items, which score near 51 in Devanagari, inflate by +7.8 (IAST), +7.4 (ASCII), and +6.5 (Hinglish) points, all $p \leq 1.5 \times 10^{-4}$. The same shape holds for Pro at smaller magnitude. This is the pattern one expects if romanization blurs the judge’s view of quality: content whose flaws are obvious stays low, content whose merits are obvious stays high, and ambiguous content drifts toward a generous middle. Figure 2 shows one such drift verbatim: one refrigerator explanation, four orthographies, and a thirty-point spread under a single judge.

Table 1: Mean within-item score shift (romanized minus Devanagari) for every judge and romanization, with bootstrap 95% confidence intervals over $n=150$ paired items. Positive means the identical content scored *higher* romanized. **Bold** marks shifts whose permutation $p < 0.01$. The three families disagree in sign: Gemini inflates everywhere, Qwen inflates formal romanizations only, and Claude penalises, most strongly at the smallest scale. All judges here are scored one item per call; Section 4.3 shows the batched alternative masks the Claude penalties.

Judge	IAST	ASCII	Hinglish
Gemini 3.6 Flash	+3.43 [+2.4, +4.6]	+2.17 [+0.8, +3.7]	+3.13 [+2.2, +4.1]
Gemini 3.1 Pro	+1.17 [+0.6, +1.8]	+1.10 [+0.4, +1.8]	+1.13 [+0.5, +1.8]
Qwen2.5-7B	+7.63 [+4.9, +10.4]	+6.03 [+3.3, +8.7]	+0.07 [-1.8, +1.9]
Claude Sonnet 5	-2.70 [-4.7, -0.8]	-12.29 [-15.6, -9.2]	-2.71 [-4.0, -1.4]
Claude Opus 5	-0.13 [-0.8, +0.6]	-3.43 [-4.4, -2.5]	-1.18 [-1.9, -0.5]
Claude Fable 5	+1.43 [+1.0, +1.9]	-0.30 [-1.1, +0.4]	+0.13 [-0.3, +0.6]

4.3 The measurement protocol can hide the bias

Our first Claude measurements used batched agent sessions: each session scored 150 items in one context, with conditions rotated in a Latin square. Under that protocol all three Claude judges appeared script-invariant, shifting at most 0.54 points in eight of nine comparisons. Rerunning the identical items one per call flipped the answer (Figure 5): the invariance was an artifact of the batch. A judge that has just scored dozens of items appears to anchor its scale on content quality and stops reacting to surface form; a judge shown one item in isolation, with nothing to anchor against, lets the orthography move the score. We report both protocols because the difference is itself a finding: batched audits, which are common because they are cheap, can mask a twelve-point bias. Judge-bias measurements should state their context protocol, and deployed pipelines should note that most production judging is one-item-per-call, the protocol under which the biases here are largest.

4.4 Prompting does not reliably fix it

The remedy every practitioner would try first is a line in the judge prompt. We tested six: the plain instruction (“evaluate the content regardless of the script or orthography”), an explicit warning naming both scripts, a fairness framing demanding identical scores for identical content, a normalization instruction (“mentally transliterate to Devanagari first”), a script-blind persona, and a decomposed rubric that forces four sub-scores summing to 100. Each ran over the full four-condition protocol on two judge families (Figure 6).

On Gemini 3.6 Flash, no arm closes the gap. The best, transliterate-first, cuts medium-tier inflation from +7.8 to +4.4; the persona arm does nothing (+7.7); and the decomposed rubric overshoots, flipping ASCII from +7.4 to -2.8, trading inflation for penalty rather than removing bias. On Qwen2.5-7B the battery *backfires* uniformly: every instruction increases IAST inflation above the +7.6 no-instruction baseline, up to +11.8 under the

fairness framing, and the script-blind persona even induces a +4.6 Hinglish inflation where the baseline judge was flat. Mentioning scripts in the prompt appears to direct the smaller judge’s attention to the very feature it should ignore. Prompt-level mitigation is therefore unreliable in the strongest sense: its effect ranges from partial reduction through sign reversal to amplification, consistent with debiasing results showing prompt-level fixes underperform training-level ones (Zhou et al., 2026b). The practitioner remedy remains judge selection and script-stratified reporting, not prompting.

4.5 Mechanism: not token length; the script is perceived

Two probes narrow the explanation space. First, tokenization. A standing hypothesis from the tokenizer-fairness literature is that length disparities drive downstream behaviour (Petrov et al., 2023; Ahia et al., 2023; Srivastava, 2026). We measured every item in every condition with the Gemini tokenizer: romanized Hindi is the *expensive* form, costing 1.98 (IAST), 1.47 (ASCII), and 1.33 (Hinglish) times the tokens of the same content in Devanagari (Figure 7, left). This is itself a finding worth recording, because it reverses the premise of the romanization-efficiency literature, which reported 2 to 4-fold token *savings* from romanization (Husain et al., 2024); modern tokenizers have since closed the Devanagari gap (Brahma et al., 2025). But token inflation does not explain the score inflation: the per-item correlation between token ratio and score shift is $r = +0.11$ (IAST), $+0.05$ (ASCII), and -0.08 (Hinglish), and no tier-restricted correlation exceeds $|r| = 0.18$. Longer input is not what earns the higher score.

Second, perception. The judge’s own rationales show it sees what it is scoring (Figure 7, right): under Gemini 3.6 Flash, 47 percent of IAST, 57 percent of ASCII, and 41 percent of Hinglish rationales explicitly mention the script or romanization, against 1 percent for Devanagari. Some rationales even name the orthography as a deficiency while the score lands higher than the Devanagari twin’s. The bias therefore operates despite awareness, consistent with the

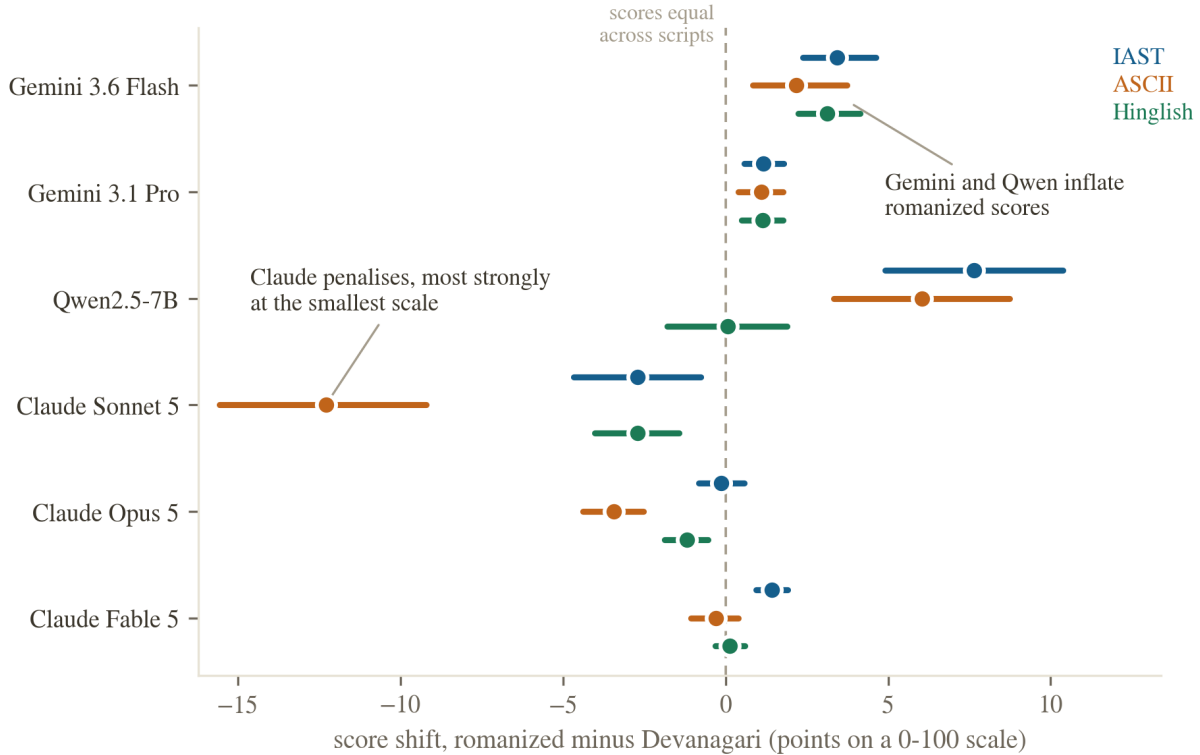


Figure 3: Score shift by judge and romanization, with 95 percent bootstrap confidence intervals. Each row group is one judge; each interval is the mean within-item shift (romanized minus Devanagari) over $n = 150$ paired items. Read intervals against the dashed zero line: right of zero means romanized content scored higher than the same content in Devanagari. Both Gemini judges and Qwen (on formal romanizations) sit right of zero; all three Claude judges, scored one item per call, sit at or left of zero, with the penalty largest for the smallest model.

mitigation failure: this is a miscalibration of quality assessment through an unfamiliar surface form, echoing at the behavioural level the finding that internal representations are not script-invariant (Karne, 2026; Saji et al., 2025).

5 Limitations, narrated

We narrate the study’s weaknesses as part of the story, because several were design choices made for speed and budget, and a reader should weigh them.

The dataset is model-authored. All 150 items and their quality tiers were generated by a language model under human-specified tier definitions, then frozen. Tier intent was strongly realised in the scores, but model-authored Hindi may differ from human Hindi in ways that interact with romanization. The hinglish condition is likewise a model respelling under a strict no-paraphrase rule; the protocol includes a 20-item native-speaker fidelity audit, and the audited sample with its verdict ships with the data.

The Claude judges were measured under two protocols: batched agent sessions and headless one-item-per-call invocations of the pinned model ids. The headline Claude numbers use the isolated protocol, which matches

the Gemini and Qwen arms; its one residual difference is that the headless interface does not expose temperature control, so Claude judgments use default sampling where the API arms use temperature 0. One of the three Claude judges shares a model family with the orchestrator of this study; the isolated protocol’s blind single-item calls mitigate, and the family’s penalty direction is opposite to the inflation a self-favouring judge would produce.

Scale and scope are modest: one language, 150 items, six judges, one seed at temperature 0. The mitigation battery covers six prompt-level arms on two judge families but no training-level intervention. Our quality tiers are coarse; a continuous-quality design would resolve the discretion story more finely. Extending the audit to other digraphic settings, such as Serbian Cyrillic and Latin (Karne, 2026) or romanized Sinhala (Rajapakse and Weerasinghe, 2026), is the natural next step.

Finally, the pre-registration was directional and informal (recorded in the project log before data collection), not a third-party registry entry. We state it because it disciplined our reporting: the headline effect is opposite to what we predicted.

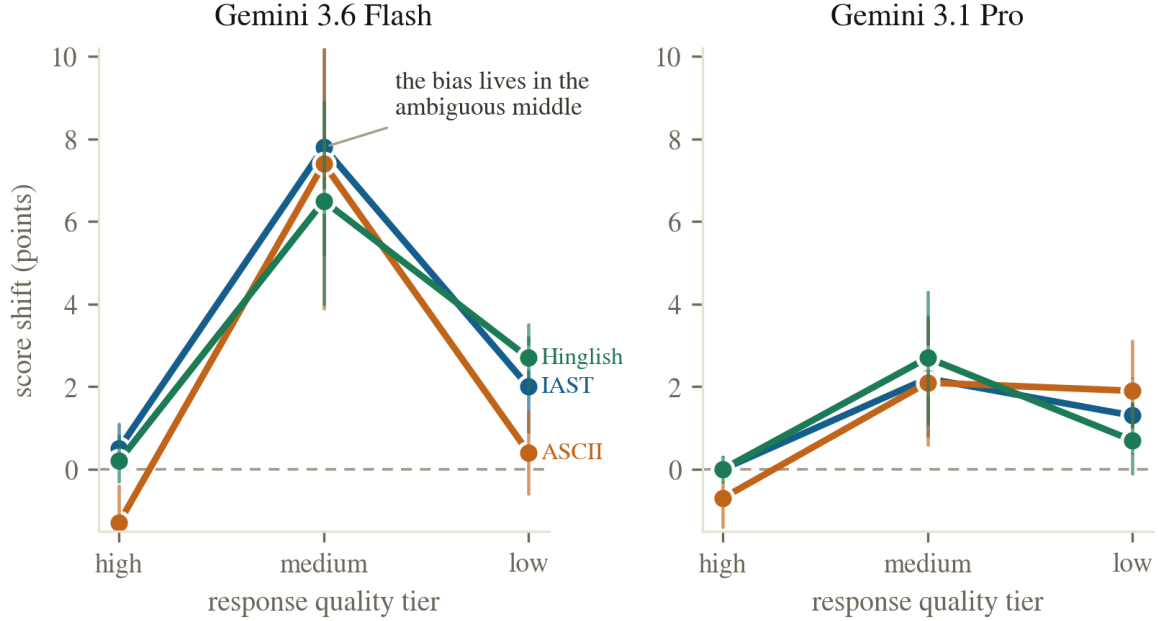


Figure 4: The bias concentrates where the judge has discretion. Mean shift by intended quality tier for the two Gemini judges, with 95 percent bootstrap intervals ($n = 50$ items per tier). High-tier items sit near the score ceiling (Devanagari mean 99.1 under Flash) and low-tier items near the floor (15.2), leaving little room to move; medium-tier items (Devanagari mean 51.4) inflate by 6.5 to 7.8 points under Flash and 2.1 to 2.7 under Pro.

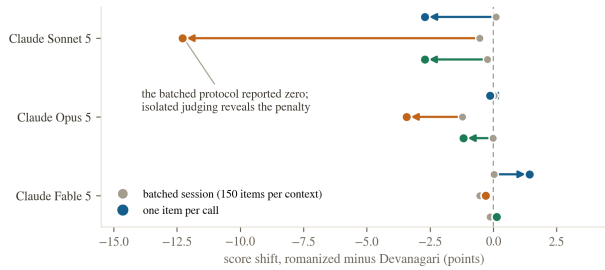


Figure 5: The same judges, two protocols, two answers. Mean shift for the three Claude judges under batched sessions (grey dots: 150 items per context, Latin-square rotation) versus one item per call (colored dots), arrows pointing from the batched to the isolated estimate. Batched judging reported near-zero shifts for all three judges; isolated judging reveals penalties as large as -12.3 points.

6 Conclusion

We held Hindi content fixed, changed only its writing system, and asked six LLM judges to grade it. The writing system moved the grades, in both directions. Gemini judges inflate romanized Hindi by 2 to 3 points overall and 7 to 8 on ambiguous-quality content; a 7B open-weights judge inflates formal romanizations by up to 18 points on low-quality content while judging natural Hinglish soberly; Claude judges tax romanization instead, by up to 12.3 points at the smallest scale, shrinking monotoni-

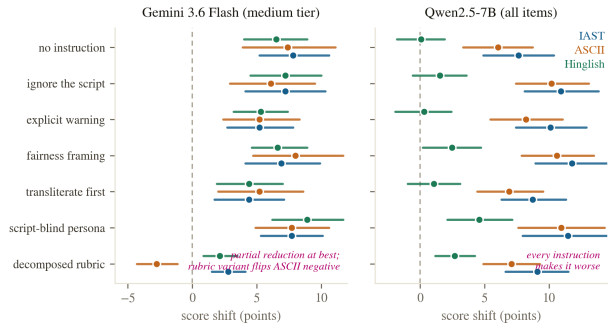


Figure 6: Six mitigation arms, two judge families, no reliable fix. Mean shift with 95 percent bootstrap CIs for each prompt intervention. **Left:** Gemini 3.6 Flash on medium-tier items ($n = 50$ per condition): reductions are partial at best and the decomposed rubric flips ASCII negative. **Right:** Qwen2.5-7B on all items ($n = 150$): every instruction increases the inflation above the no-instruction baseline.

cally with capability; the batched measurement protocol that is standard in judge audits masks the entire Claude effect; six prompt-level mitigations range from partial to actively counterproductive; token length does not explain any of it; and the judges’ own rationales prove the script is seen. The bias axis is real, it is judge-dependent in size and sign, it is protocol-sensitive, and it belongs in every judge-bias taxonomy (Ye et al., 2025; Zhou et al., 2026b) and every multilingual evaluation checklist (Hada

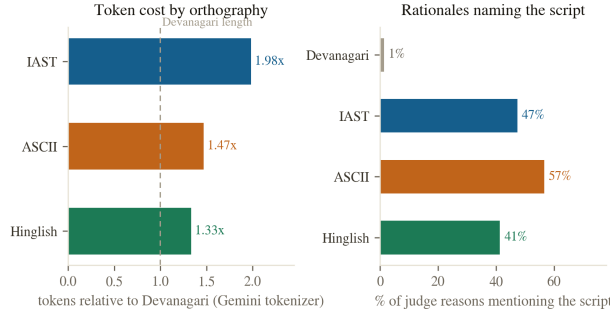


Figure 7: Left: the same content costs 1.33 to 1.98 times as many tokens romanized as in Devanagari under the Gemini tokenizer (mean over 150 items; Devanagari mean 93 tokens). **Right:** share of the judge’s one-sentence rationales that mention script, romanization, or transliteration, by condition, under Gemini 3.6 Flash ($n = 150$ per condition). The judge names the script on 41 to 57 percent of romanized items and on 1 percent of Devanagari items.

et al., 2024b; Son et al., 2024). Until judges are audited for script invariance, evaluations of Hindi systems should report native-script and romanized slices separately, and should say which judge produced the numbers.

Reproducibility

All datasets, per-call score files, the scripted analysis, figure build scripts, and spend logs are released at <https://github.com/Abraar237/script-bias-llm-judges>, with a narrative project page at <https://abraar237.github.io/script-bias-llm-judges/>. Every number above traces to `results/analysis.json`, generated by `experiments/analyze.py` from the raw judgment files. Total marginal compute cost of the study: under 5 US dollars of API spend plus roughly one A10G GPU-hour.

References

Orevaoghene Ahia, Sachin Kumar, Hila Gonen, Jungo Kasai, David R. Mortensen, Noah A. Smith, and Yulia Tsvetkov. Do all languages cost the same? Tokenization in the era of commercial language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

Kabir Ahuja, Harshita Diddee, Rishav Hada, Millicent Ochieng, Krithika Ramesh, Prachi Jain, Akshay Nambi, Tanuja Ganu, Sameer Segal, Maxamed Axmed, Kalika Bali, and Sunayana Sitaram. MEGA: Multilingual evaluation of generative AI. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

Mehdi Ali, Michael Fromm, Klaudia Thellmann, Richard Rutmann, Max Lübbering, Johannes Leveling, Katrin Klug, Jan Ebert, Niclas Doll, Jasper Schulze Buschhoff, Charvi Jain, Alexander Arno Weber, Lena Jurkschat, Hammam Abdelwahab, Chelsea John, Pedro Ortiz Suarez, Malte Ostendorff, Samuel Weinbach, Rafet Sifa, Stefan Kesselheim, and Nicolas Flores-Herr. Tokenizer choice for LLM training: Negligible or crucial? In *Findings of the Association for Computational Linguistics: NAACL 2024*, 2024.

Maharaj Brahma, N J Karthika, Atul Singh, Devaraj Adiga, Smruti Bhate, Ganesh Ramakrishnan, Rohit Saluja, and Maunendra Sankar Desarkar. MorphTok: Morphologically grounded tokenization for indian languages. In *Tokenization Workshop (TokShop) at ICML 2025*, 2025.

Avisha Das, Mihir Parmar, Mohana Ramnath, and Pulkit Verma. Indi-RomCoM: Code-mixed benchmark for evaluating LLMs on romanized Indic-English instructions. *arXiv preprint arXiv:2606.30790*, 2026.

Sumanth Doddapaneni, Rahul Aralikatte, Gowtham Ramesh, Shreya Goyal, Mitesh M. Khapra, Anoop Kunchukuttan, and Pratyush Kumar. Towards leaving no indic language behind: Building monolingual corpora, benchmark and models for indic languages. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2023.

Xiyan Fu and Wei Liu. How reliable is multilingual LLM-as-a-Judge? In *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025.

Jay Gala, Thanmay Jayakumar, Jaavid Aktar Husain, Aswanth Kumar M, Mohammed Safi Ur Rahman Khan, Diptesh Kanojia, Ratish Puduppully, Mitesh M. Khapra, Raj Dabre, Rudra Murthy, and Anoop Kunchukuttan. Airavata: Introducing Hindi instruction-tuned LLM. *arXiv preprint arXiv:2401.15006*, 2024.

Rishav Hada, Varun Gumma, Mohamed Ahmed, Kalika Bali, and Sunayana Sitaram. METAL: Towards multilingual meta-evaluation. In *Findings of the Association for Computational Linguistics: NAACL 2024*, 2024a.

Rishav Hada, Varun Gumma, Adrian de Wynter, Harshita Diddee, Mohamed Ahmed, Monojit Choudhury, Kalika Bali, and Sunayana Sitaram. Are large language model-based evaluators the solution to scaling up multilingual evaluation? In *Findings of the Association for Computational Linguistics: EACL 2024*, 2024b.

Jaavid Aktar Husain, Raj Dabre, Aswanth Kumar, Jay Gala, Thanmay Jayakumar, Ratish Puduppully, and Anoop Kunchukuttan. RomanSetu: Efficiently unlocking multilingual capabilities of large language models via romanization. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024.

- Sripad Karne. One language, two scripts: Probing script-invariance in LLM concept representations. *arXiv preprint arXiv:2603.08869*, 2026.
- Shreyas KC. BabelJudge: Measuring LLM-as-a-Judge reliability across languages and agent trajectories. *arXiv preprint arXiv:2606.22329*, 2026.
- Manurag Khullar, Utkarsh Desai, Poorva Malviya, Aman Dalmia, and Zheyuan Ryan Shi. Script gap: Evaluating LLM triage on indian languages in native vs romanized scripts in a real world setting. *arXiv preprint arXiv:2512.10780*, 2025.
- Davis Liang, Hila Gonen, Yuning Mao, Rui Hou, Naman Goyal, Marjan Ghazvininejad, Luke Zettlemoyer, and Madian Khabsa. XLM-V: Overcoming the vocabulary bottleneck in multilingual masked language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.
- Tomasz Limisiewicz, Terra Blevins, Hila Gonen, Orevaoghene Ahia, and Luke Zettlemoyer. MYTE: Morphology-driven byte encoding for better and fairer multilingual language modeling. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024.
- Yash Madhani, Sushane Parthan, Priyanka Bedekar, Gokul NC, Ruchi Khapra, Anoop Kunchukuttan, Pratyush Kumar, and Mitesh M. Khapra. Aksharantar: Open indic-language transliteration datasets and models for the next billion users. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023.
- Arjun Panickssery, Samuel R. Bowman, and Shi Feng. LLM evaluators recognize and favor their own generations. In *Advances in Neural Information Processing Systems 37*, 2024.
- Aleksandar Petrov, Emanuele La Malfa, Philip H. S. Torr, and Adel Bibi. Language model tokenizers introduce unfairness between languages. In *Advances in Neural Information Processing Systems 36*, 2023.
- Sukannya Purkayastha, Sebastian Ruder, Jonas Pfeiffer, Iryna Gurevych, and Ivan Vulić. Romanization-based large-scale adaptation of multilingual language models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023.
- Minuri Rajapakse and Ruwan Weerasinghe. Script sensitivity: Benchmarking language models on Unicode, romanized and mixed-script Sinhala. *arXiv preprint arXiv:2601.14958*, 2026.
- Alan Saji, Jaavid Aktar Husain, Thanmay Jayakumar, Raj Dabre, Anoop Kunchukuttan, and Ratish Puduppully. RomanLens: The role of latent romanization in multilinguality in LLMs. In *Findings of the Association for Computational Linguistics: ACL 2025*, 2025.
- Rajvee Sheth, Himanshu Beniwal, and Mayank Singh. COMI-LINGUA: Expert annotated large-scale dataset for multitask NLP in Hindi-English code-mixing. *arXiv preprint arXiv:2503.21670*, 2025.
- Harman Singh, Nitish Gupta, Shikhar Bharadwaj, Dinesh Tewari, and Partha Talukdar. IndicGenBench: A multilingual benchmark to evaluate generation capabilities of LLMs on indic languages. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024.
- Guijin Son, Dongkeun Yoon, Juyoung Suk, Javier Aula-Blasco, Mano Aslan, Vu Trong Kim, Shayekh Bin Islam, Jaume Prats-Cristià, Lucia Tormo-Bañuelos, and Seungone Kim. MM-Eval: A multilingual meta-evaluation benchmark for LLM-as-a-Judge and reward models. *arXiv preprint arXiv:2410.17578*, 2024.
- Priyansh Srivastava. The tokenizer tax: Quantifying and explaining the cross-lingual cost of subword tokenization for indian languages. *arXiv preprint arXiv:2607.24276*, 2026.
- Sshubam Verma, Mohammed Safi Ur Rahman Khan, Vishwajeet Kumar, Rudra Murthy, and Jaydeep Sen. MILU: A multi-task indic language understanding benchmark. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2025.
- Peiyi Wang, Lei Li, Liang Chen, Zefan Cai, Dawei Zhu, Binghuai Lin, Yunbo Cao, Qi Liu, Tianyu Liu, and Zhi-fang Sui. Large language models are not fair evaluators. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024.
- Ishaan Watts, Varun Gumma, Aditya Yadavalli, Vivek Shashadri, Manohar Swaminathan, and Sunayana Sitaram. PARIKSHA: A large-scale investigation of human-LLM evaluator agreement on multilingual and multi-cultural data. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024.
- Orgest Xhelili, Yihong Liu, and Hinrich Schütze. Breaking the script barrier in multilingual pre-trained language models with transliteration-based post-training alignment. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024.
- Yilun Yang and Yekun Chai. CodeMixBench: Evaluating code-mixing capabilities of LLMs across 18 languages. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025.
- Jiayi Ye, Yanbo Wang, Yue Huang, Dongping Chen, Qi-hui Zhang, Nuno Moniz, Tian Gao, Werner Geyer, Chao Huang, Pin-Yu Chen, Nitesh V. Chawla, and Xiangliang Zhang. Justice or prejudice? Quantifying biases in LLM-as-a-Judge. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.

- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36, Datasets and Benchmarks Track*, 2023.
- Ej Zhou, Lucas Resck, Zheng Hui, and Anna Korhonen. Lower-resource, higher scores: Language bias in LLM evaluators. *arXiv preprint arXiv:2607.14480*, 2026a.
- Hongli Zhou, Hui Huang, Rui Zhang, Kehai Chen, Bing Xu, Conghui Zhu, Tiejun Zhao, and Muyun Yang. Toward robust LLM-based judges: Taxonomic bias evaluation and debiasing optimization. *arXiv preprint arXiv:2603.08091*, 2026b.
- Irune Zubiaga, Aitor Soroa, and Rodrigo Agerri. Towards reliable multilingual LLMs-as-a-Judge: An empirical study. *arXiv preprint arXiv:2605.28710*, 2026.